

使用 CrewAI、LangGraph、A2A 和 ADK 擴充代理

來源：<https://codelabs.developers.google.com/next26/scale-agents?hl=zh-tw>

步驟數：11

瞭解如何使用 CrewAI、LangGraph、A2A 通訊協定和 ADK，透過 Vertex AI 擴充代理。

目錄

1. [1. 簡介](#)
2. [2. 事前準備](#)
3. [3. 定義工具和代理程式](#)
4. [4. 定義工作和工作人員](#)
5. [5. 建立 LangGraph 規劃工具](#)
6. [6. 執行 Planner 和 Crew](#)
7. [7. 將規劃工具封裝到 A2A 伺服器中](#)
8. [8. 建構 ADK 控制室](#)
9. [9. 執行完整堆疊](#)
10. [10. 清除所用資源](#)
11. [11. 恭喜](#)

1. 簡介

在本程式碼實驗室中，您將瞭解如何使用 CrewAI、LangGraph、A2A 通訊協定和 ADK (Agent Development Kit)，建構多代理自動化調度管理系統。您將建立一個系統，讓 ADK 控制室將規劃工作委派給 LangGraph 規劃工具，再由該工具將工作指派給 CrewAI 執行團隊 (所有環節都透過 A2A 連線)，處理零售庫存補貨情境。

什麼是多代理自動化調度管理？

在多代理系統中，多個專業 AI 代理會協同合作，完成單一代理無法處理的複雜任務。您不必使用單一的巨型代理程式處理所有事項，而是將問題分解為規劃者和執行者這兩個角色，每個角色都有自己的工具和專業知識。

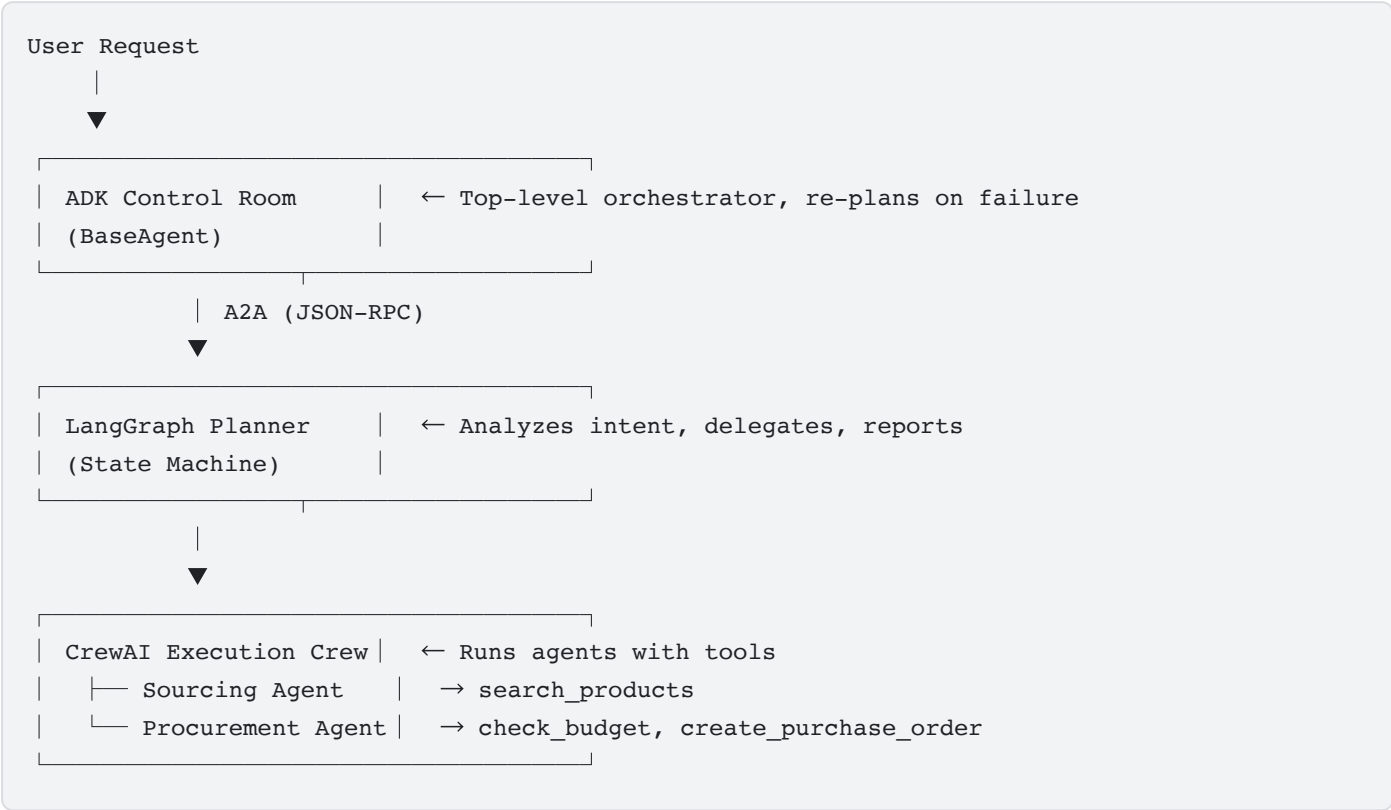
這與人類組織的運作方式類似：經理將策略委派給分析師，並將執行作業委派給專家。這樣的好處包括：

- **關注點分離**：每個代理專注於自己最擅長的工作
- **框架彈性**：為每個角色使用最合適的框架 (LangGraph 用於規劃邏輯，CrewAI 用於執行工具)
- **可擴充性**：新增專用代理程式，不必變更整個系統

情境

當使用者傳送「Restock 1 Pixel 7 phone for the Tokyo office」(為東京辦公室補貨 1 支 Pixel 7 手機) 這類補貨要求時，系統會：

1. **LangGraph Planner** 會分析要求，並擷取項目和數量
2. 規劃工具會將執行作業委派給 **CrewAI 執行團隊**
3. **採購專員**代理程式會使用工具搜尋產品目錄
4. **採購人員**代理會驗證預算，並使用工具下達採購單
5. 結果會回流至規劃工具，並產生最終報表



技術堆疊

圖層	科技	角色
規劃	LangGraph	分析意圖、轉送要求、產生報表的狀態機器
執行作業	CrewAI	依序呼叫工具的角色型代理程式
LLM	Vertex AI 的 Gemini	支援代理推論和工具選取
代理程式間的通訊	A2A 通訊協定	JSON-RPC 2.0 橋接器，讓不同架構的代理程式可以通訊
頂層自動化調度管理工具	ADK (BaseAgent)	接收要求、透過 A2A 委派，並在失敗時重新規劃

實際運作情況：如有可用的完整正式版系統，請前往 <https://scale-control-room-761793285222.us-central1.run.app> 試用。這個系統會擴充您在此建構的內容，提供即時資訊主頁、A2A 通訊協定和 IAM 安全性。

學習內容

- 定義代理程式可使用的自訂工具。
- 使用 **CrewAI** 建構專用代理。
- 使用 **LangGraph** 建立狀態機規劃工具。
- 協調規劃人員和執行團隊之間的流程。
- 將規劃工具包裝在 **A2A 通訊協定** 伺服器中，以進行跨架構通訊。
- 建立頂層 **ADK 控制室**，透過 A2A 委派，並在失敗時重新規劃。

軟硬體需求

- 網路瀏覽器，例如 [Chrome](#)
- 已啟用計費功能的 Google Cloud 雲端專案

本程式碼研究室適合**中階**開發人員，他們熟悉 Python 和基本 LLM 概念。

預計時間：**35 分鐘**。

費用預估：本程式碼研究室建立的資源費用應不到 \$1 美元。

步驟 2 · 約 3 分鐘

2. 事前準備

info

從做中學 免付費

立即取得 Google Cloud 抵免額，並完成這個實驗室。無須提供信用卡或付款方式。

啟用

建立 Google Cloud 專案

1. 在 [Google Cloud 控制台](#) 的專案選取器頁面中，[選取或建立 Google Cloud 專案](#)。
2. 確認 Cloud 專案已啟用計費功能。瞭解如何[檢查專案是否已啟用計費功能](#)。

啟動 Cloud Shell

Cloud Shell 是在 Google Cloud 中運作的指令列環境，已預先載入必要工具。

1. 點選 Google Cloud 控制台頂端的「啟用 Cloud Shell」。
2. 連至 Cloud Shell 後，請驗證您的驗證：

```
gcloud auth list
```

3. 確認專案已設定完成：

```
gcloud config get project
```

4. 如果專案未如預期設定，請設定專案：

```
export PROJECT_ID=<YOUR_PROJECT_ID>
gcloud config set project $PROJECT_ID
```

啟用 API

執行下列指令來啟用 Vertex AI API：

```
gcloud services enable aiplatform.googleapis.com
```

注意：Cloud Shell 會自動透過您的 Google Cloud 帳戶進行驗證。如果您是在 Cloud Shell 以外的地方執行這個程式碼實驗室，請執行 `gcloud auth application-default login`，向 Vertex AI 進行驗證。

設定環境

在 Cloud Shell 中，為專案建立新目錄並前往該目錄：

```
mkdir scale-agents
cd scale-agents
```

安裝 `uv`，並使用該工具安裝必要套件：

```
curl -LsSf https://astral.sh/uv/install.sh | sh
export PATH="$HOME/.local/bin:$PATH"
uv init --no-workspace --no-readme
rm main.py
sed -i 's/requires-python = ">=3.12"/requires-python = ">=3.12,<3.14"/' pyproject.toml
uv add crewai 'litellm[google]' langgraph 'a2a-sdk>=0.3.25,<0.4' httpx uvicorn 'google-adk>=1.0.0' --prerelease=allow
```

設定 Vertex AI 的環境變數：

```
export GOOGLE_CLOUD_PROJECT=$(gcloud config get-value project)
export GOOGLE_CLOUD_LOCATION=us-central1
export GOOGLE_GENAI_USE_VERTEXAI=TRUE
```

步驟 3 · 約 5 分鐘

3. 定義工具和代理程式

在多代理系統中，代理需要工具與外界互動，以及特定角色來瞭解該做什麼。

建立名為 `scale_agents.py` 的檔案，並加入下列程式碼。這會設定匯入內容、模擬工具和 CrewAI 代理程式。

```

import os
from typing import Optional
from crewai import Agent, Crew, Process, Task, LLM
from crewai.tools import tool
from langgraph.graph import StateGraph, END
from typing_extensions import TypedDict

# CrewAI requires this env var even when using Vertex AI
os.environ["OPENAI_API_KEY"] = "NA"

# Set the project ID for Vertex AI
os.environ["VERTEXAI_PROJECT"] = os.getenv("GOOGLE_CLOUD_PROJECT", "")
os.environ["VERTEXAI_LOCATION"] = "us-central1"

# Initialize the LLM to use Vertex AI
llm = LLM(
    model="vertex_ai/gemini-2.5-flash",
    temperature=0.0,
    max_tokens=4096,
)

# --- Step 1: Define Tools ---

@tool("search_products")
def search_products(query: str) -> list:
    """Search for products in the catalog."""
    # Mock product catalog
    products = [
        {"product_id": "pixel-7", "name": "Pixel 7 Phone", "price": 50.0},
        {"product_id": "pixel-8", "name": "Pixel 8 Phone", "price": 80.0},
    ]
    return [p for p in products if query.lower() in p["name"].lower()]

@tool("check_budget")
def check_budget(amount: float) -> dict:
    """Check if a purchase amount is within the budget."""
    limit = 100.0
    if amount <= limit:
        return {"approved": True, "remaining": limit - amount}
    return {"approved": False, "reason": f"Exceeds budget of ${limit}"}

@tool("create_purchase_order")
def create_purchase_order(product_id: str, quantity: int) -> dict:
    """Create a purchase order for a product."""
    return {
        "status": "SUCCESS",
        "po_id": f"PO-{product_id}-{quantity}",
        "message": f"Successfully ordered {quantity} units of {product_id}."
    }

# --- Step 2: Define Agents ---

```



```
sourcing_agent = Agent(
    role="Sourcing Specialist",
    goal="Find the best available products that match the intent of the request. You MUST use
the search_products tool to look up products -- never make up product data.",
    backstory="You are a veteran procurement specialist with an eye for detail. You always
search the catalog before recommending a product.",
    tools=[search_products],
    llm=llm,
    verbose=True,
    allow_delegation=False,
    memory=False,
    reasoning=False,
)

procurement_agent = Agent(
    role="Procurement Officer",
    goal="Validate the purchase against budget constraints and execute the order. You MUST
use the check_budget tool before ordering and the create_purchase_order tool to place the
order.",
    backstory="You are the gatekeeper of the budget. You always verify budget before placing
any order.",
    tools=[check_budget, create_purchase_order],
    llm=llm,
    verbose=True,
    allow_delegation=False,
    memory=False,
    reasoning=False,
)
```

核心概念

- **@tool 裝飾器**：CrewAI 會使用這個裝飾器，將一般 Python 函式轉換為 LLM 可瞭解及呼叫的工具。函式的型別提示和 docstring 會用於產生 LLM 可理解的工具結構定義。
- **角色、目標和背景故事**：這些內容會定義代理的角色，並引導 LLM 推論。背景故事不只是裝飾文字，「你一律會搜尋目錄」可鼓勵代理程式使用工具，而不是產生幻覺。
- **reasoning=False**：停用擴展推論，讓代理程式遵循標準工具呼叫迴圈，而不是嘗試直接回答。
- **allow_delegation=False**：讓每個代理專注於自己分配到的工具，而不是將工作傳遞給其他代理。

為什麼需要兩個代理程式，而不是一個？每位服務專員都有不同的工具和工作。採購專員只負責搜尋產品，採購人員則只負責處理預算和訂單。這種**關注點分離**的做法，可確保每個代理程式都有專屬提示和一組相關工具，相較於單一代理程式處理所有事項，LLM 行為更可靠。

4. 定義工作和工作人員

現在，我們來建立工作，並將這些工作連線至團隊，定義這些代理程式需要執行的動作。

在同一個 `scale_agents.py` 檔案結尾處附加下列程式碼：

```
# --- Step 3: Define Tasks & Crew ---

sourcing_task = Task(
    description="Use the search_products tool to find products matching:
    '{item_description}'. Return the product_id and price of the best match from the tool
    results.",
    expected_output="The product_id and price of the best matching product from the
    search_products tool.",
    agent=sourcing_agent
)

procurement_task = Task(
    description="First, use the check_budget tool to verify the total cost for {quantity}
    units. Then use the create_purchase_order tool with the product_id and quantity to place the
    order.",
    expected_output="The purchase order details returned by the create_purchase_order tool.",
    agent=procurement_agent,
    context=[sourcing_task] # This task depends on the output of sourcing_task
)

def run_crew(item_description: str, quantity: int):
    crew = Crew(
        agents=[sourcing_agent, procurement_agent],
        tasks=[sourcing_task, procurement_task],
        process=Process.sequential, # Run tasks in order
        verbose=True,
        memory=False,
        planning=False,
    )

    result = crew.kickoff(inputs={
        "item_description": item_description,
        "quantity": quantity
    })
    return result
```

核心概念

- **工作內容**： `context=[sourcing_task]` 告訴 CrewAI，採購工作需要先完成採購工作才能繼續。採購人員可以查看採購專員的發現，再決定要訂購哪些產品。
- **Process.sequential**：工作會按照列出的順序執行。這項步驟非常重要，因為採購工作取決於採購任務的結果，您必須先知道要購買的產品，才能下單。

- `memory=False` / `planning=False`：停用 CrewAI 的內建記憶體和規劃功能，確保執行作業簡單且可預測，以利進行這項示範。
-

步驟 5 · 約 5 分鐘

5. 建立 LangGraph 規劃工具

執行團隊負責「如何」執行，包括搜尋產品、檢查預算、下單等。但誰決定「什麼」？這是使用 **LangGraph** 建構的**規劃代理**。

LangGraph 會將工作流程建模為**狀態機**，也就是由邊緣 (轉換) 連接的節點 (函式) 圖表。狀態會流經圖表，每個節點都會讀取及寫入共用狀態。這非常適合規劃工作流程，因為您需要明確的確定性控制流程：分析要求、委派給工作人員、產生報表。

在同一個 `scale_agents.py` 檔案結尾處附加下列程式碼：

```
# --- Step 4: Define LangGraph Planner ---

class PlanState(TypedDict):
    objective: str
    item_description: Optional[str]
    quantity_needed: Optional[int]
    execution_result: Optional[str]
    final_report: Optional[str]

def analyze_alert(state: PlanState) -> PlanState:
    """Node 1: Extract intent from the raw objective string."""
    print("--- ANALYZING ALERT ---")
    # In a production app, you would use an LLM here to extract details.
    # For simplicity, we simulate extraction here.
    objective = state["objective"]

    # Hardcoded extraction for the demo
    if "Pixel 7" in objective:
        return {
            "item_description": "Pixel 7",
            "quantity_needed": 1,
        }
    return {
        "item_description": "unknown",
        "quantity_needed": 0,
    }

def delegate_to_executor(state: PlanState) -> PlanState:
    """Node 2: Call the CrewAI Execution Crew."""
    print("--- DELEGATING TO CREW ---")
    if state["item_description"] == "unknown":
        return {"execution_result": "Failed: Unknown item"}

    result = run_crew(
        item_description=state["item_description"],
        quantity=state["quantity_needed"]
    )
    return {"execution_result": str(result)}

def generate_report(state: PlanState) -> PlanState:
    """Node 3: Synthesize the final outcome."""
    print("--- GENERATING REPORT ---")
    return {
        "final_report": f"Objective handled: {state['objective']}. Result: {state['execution_result']}"
    }

# Build the graph
workflow = StateGraph(PlanState)
workflow.add_node("analyze_alert", analyze_alert)
workflow.add_node("delegate", delegate_to_executor)
workflow.add_node("generate_report", generate_report)
```

```
workflow.set_entry_point("analyze_alert")
workflow.add_edge("analyze_alert", "delegate")
workflow.add_edge("delegate", "generate_report")
workflow.add_edge("generate_report", END)

app = workflow.compile()

# --- Main Execution ---
if __name__ == "__main__":
    print("Starting Multi-Agent System...")

    initial_state = {
        "objective": "Restock 1 Pixel 7 phones for the Tokyo office"
    }

    final_state = app.invoke(initial_state)

    print("\n=== FINAL REPORT ===")
    print(final_state["final_report"])
```

核心概念

- **StateGraph**：定義狀態機器。PlanState 是型別狀態，會在每個節點處理要求時累積。
- **節點**：函式會採用目前狀態，並傳回更新內容。每個節點都只負責一項工作：`analyze_alert` 擷取意圖、`delegate_to_executor` 執行工作人員、`generate_report` 總結結果。
- **邊緣**：定義節點之間的流程。在本程式碼研究室中，我們使用簡單的線性流程 (`analyze` → `delegate` → `report`)。完整研討會會使用**條件式路徑**擴充此流程，例如將破壞性要求路徑導向安全路徑，而非執行器。

為什麼要使用 **LangGraph 規劃工具**？CrewAI 非常適合用於工具呼叫代理程式，但規劃工具需要確定性的控制流程，例如「如果具有破壞性，請前往安全路徑；否則請委派」。LangGraph 的狀態機模型會明確指出這項路徑並進行測試，而 CrewAI 則會處理下方的任意形式工具執行作業。

步驟 6 · 約 3 分鐘

6. 執行 Planner 和 Crew

現在，我們來一起測試 LangGraph 規劃工具和 CrewAI 團隊。

在 Cloud Shell 終端機中執行指令碼：

```
uv run python scale_agents.py
```

輸出內容應該會指出執行的步驟：

1. **分析快訊**：執行 LangGraph 節點。
2. **委派給 Crew**：LangGraph 節點會呼叫 CrewAI 團隊。
3. **CrewAI 執行**：你會看到採購專員搜尋產品，以及採購人員檢查預算並建立訂購單。
4. **最終報告**：最後會列印匯總結果。

輸出內容範例 (已縮寫)：

```
Starting Multi-Agent System...
--- ANALYZING ALERT ---
--- DELEGATING TO CREW ---

Agent: Sourcing Specialist
Tool: search_products Args: {'query': 'Pixel 7'}
Tool Completed Output: [{'product_id': 'pixel-7', 'name': 'Pixel 7 Phone', 'price': 50.0}]

Agent: Procurement Officer
Tool: check_budget Args: {'amount': 50}
Tool: create_purchase_order Args: {'product_id': 'pixel-7', 'quantity': 1}
Tool Completed Output: {'status': 'SUCCESS', 'po_id': 'PO-pixel-7-1', ...}

--- GENERATING REPORT ---

=== FINAL REPORT ===
Objective handled: Restock 1 Pixel 7 phones for the Tokyo office. Result: ...PO-pixel-7-1...SUCCESS...
```

注意：輸出內容中可能會顯示 `[CrewAIEventsBus] Warning: Event pairing mismatch` 訊息。這是 CrewAI 內部事件追蹤的表面警告，可以放心忽略。

注意：CrewAI 可能會顯示追蹤功能已停用的訊息。這項資訊僅供參考，您可以放心忽略。

注意：模擬 OMS 的預算上限為**\$100 美元**。為確保順利完成流程，請將數量保持在少量 (約 2 個單位以下)。舉例來說，1 部 Pixel 7 手機的價格為 \$50 美元，通過預算檢查；但 3 部手機的價格為 \$150 美元，會因「超出預算」而遭拒。

7. 將規劃工具封裝到 A2A 伺服器中

LangGraph 規劃工具可以運作，但會困在 Python 程序中。如要讓其他代理 (可能以不同架構編寫或在不同機器上執行) 呼叫該代理，請將其包裝在 **A2A (代理對代理)** 伺服器中。

A2A 是以 JSON-RPC 2.0 為基礎的通訊協定，可規範代理的通訊方式。重要概念：

概念	目的
代理商資訊卡	說明代理程式功能的 JSON 中繼資料 (在 <code>/.well-known/agent-card.json</code> 提供)
<code>message/send</code>	將工作傳送至代理程式的 JSON-RPC 方法
工作	工作單元 (狀態：已提交 → 處理中 → 已完成/失敗)
構件	附加至工作的中間和最終輸出內容

建立新檔案 `a2a_planner.py`：


```

import asyncio
import os
import uvicorn

from a2a.server.agent_execution import AgentExecutor, RequestContext
from a2a.server.apps import A2AStarletteApplication
from a2a.server.events import EventQueue
from a2a.server.request_handlers import DefaultRequestHandler
from a2a.server.tasks import InMemoryTaskStore, TaskUpdater
from a2a.types import (AgentCapabilities, AgentCard, AgentSkill,
                       InternalError, Part, TextPart)
from a2a.utils import new_task
from a2a.utils.errors import ServerError

# Import the LangGraph planner from Step 4
from scale_agents import app as planner_app

class PlannerAgentExecutor(AgentExecutor):
    """Wraps the LangGraph planner as an A2A service."""

    SUPPORTED_CONTENT_TYPES = ["text", "text/plain"]

    async def execute(self, context: RequestContext, event_queue: EventQueue):
        objective = context.get_user_input()

        # Initialize A2A task tracking
        task = context.current_task or new_task(context.message)
        await event_queue.enqueue_event(task)
        updater = TaskUpdater(event_queue, task.id, task.context_id)

        try:
            # Run the LangGraph planner synchronously in a thread
            initial_state = {"objective": objective}
            result = await asyncio.to_thread(planner_app.invoke, initial_state)
            final_report = result.get("final_report", "No report generated.")
        except Exception as e:
            final_report = f"Execution failed: {e}"

        # Send the result back as an artifact
        await updater.add_artifact(
            [Part(root=TextPart(text=final_report))],
            name="orchestration_report"
        )
        await updater.complete()

    async def cancel(self, context, event_queue):
        raise ServerError(error=InternalError(message="Cancel not supported"))

# Define the Agent Card – this is what other agents see
port = int(os.environ.get("PORT", 8080))

```

```

agent_card = AgentCard(
    name="Retail-Planner-A2A",
    description="LangGraph planner that delegates logistics tasks to a CrewAI crew.",
    url=f"http://localhost:{port}/",
    version="1.0.0",
    default_input_modes=PlannerAgentExecutor.SUPPORTED_CONTENT_TYPES,
    default_output_modes=PlannerAgentExecutor.SUPPORTED_CONTENT_TYPES,
    capabilities=AgentCapabilities(streaming=False),
    skills=[
        AgentSkill(
            id="plan_logistics",
            name="Plan Logistics",
            description="Analyzes inventory alerts and orchestrates procurement.",
            tags=["logistics", "planning"],
            examples=["Restock 1 Pixel 7 phones for the Tokyo office"],
        )
    ],
)

if __name__ == "__main__":
    executor = PlannerAgentExecutor()
    handler = DefaultRequestHandler(
        agent_executor=executor, task_store=InMemoryTaskStore()
    )
    server = A2AStarletteApplication(
        agent_card=agent_card, http_handler=handler
    )
    print(f"Starting A2A Planner Server on port {port}...")
    uvicorn.run(server.build(), host="0.0.0.0", port=port)

```

核心概念

- **代理資訊卡**：在 `./well-known/agent-card.json` 提供，任何代理都可以擷取該網址，瞭解這個伺服器的用途。其中列出代理程式的技能、支援的內容類型和功能。
- **`AgentExecutor.execute()`**：您實作的唯一方法。這個函式會接收傳入的要求、執行代理程式邏輯 (這裡是指 LangGraph 規劃工具)，並將結果做為構件傳回。
- **`TaskUpdater`**：管理工作生命週期，包括 `add_artifact()` 傳送中繼/最終輸出內容，以及 `complete()` 將工作標示為完成。A2A 程式庫會處理所有 JSON-RPC 管道。

在終端機中啟動 A2A 伺服器，藉此測試伺服器：

```
uv run python a2a_planner.py
```

開啟另一個 Cloud Shell 分頁 (點選目前分頁旁邊的 **+**)，並確認系統提供 Agent Card：

```
cd ~/scale-agents
curl http://localhost:8080/.well-known/agent-card.json | python3 -m json.tool
```

您應該會看到代理資訊卡 JSON。請讓第一個終端機中的 A2A 伺服器保持運作，以利進行下一個步驟。

步驟 8 · 約 5 分鐘

8. 建構 ADK 控制室

堆疊頂端是 **Control Room**，以 [ADK](#) (Google 的 Agent Development Kit) 建構而成。這項服務會接收使用者的要求、透過 A2A 委派給規劃工具、評估結果，以及處理失敗時的重新規劃 (CUJ 2)。

ADK 提供 `BaseAgent`、`LlmAgent` 和 `InMemoryRunner` 等代理原始物件。我們將 `BaseAgent` 子類別化，以編寫自訂協調邏輯，包括 A2A 呼叫、報表分類，以及使用 `LlmAgent` 子代理程式動態重新規劃。

建立新檔案 `control_room.py`：

```

import asyncio
import uuid
import os
import httpx

from google.adk.agents import BaseAgent, LlmAgent, RunConfig
from google.adk.agents.invocation_context import InvocationContext
from google.adk.events import Event
from google.adk.runners import InMemoryRunner
from google.genai import types
from typing import AsyncGenerator

A2A_SERVER_URL = os.environ.get("PLANNER_AGENT_URL", "http://127.0.0.1:8080")

def _classify_report(report: str) -> tuple[bool, bool]:
    """Return (is_success, should_retry) for a planner report."""
    normalized = (report or "").replace("*", "").strip().lower()

    success_markers = [
        "status: success", "'status': 'success'",
        "outcome: success", "po_id", "successfully ordered",
    ]
    retryable_markers = ["not found", "discontinued", "no inventory",
                        "unknown item"]
    terminal_markers = [
        "status: failed", "over budget", "not issued",
        "procurement failed",
    ]

    if any(m in normalized for m in terminal_markers):
        return False, False      # Failed, don't retry
    if any(m in normalized for m in retryable_markers):
        return False, True       # Failed, but retryable
    if any(m in normalized for m in success_markers):
        return True, False       # Success!
    return False, False         # Unknown → treat as failure

class ControlRoomAgent(BaseAgent):
    """Top-level orchestrator: delegates via A2A, re-plans on failure."""

    async def _run_async_impl(
        self, ctx: InvocationContext
    ) -> AsyncGenerator[Event, None]:
        # Extract user input from session events
        user_msg = ""
        if ctx.session and ctx.session.events:
            for evt in reversed(ctx.session.events):
                if evt.content and evt.content.role == "user":
                    user_msg = evt.content.parts[0].text
                    break

```

```

max_attempts = 2
current_objective = user_msg
final_report = "No report returned."

for attempt in range(1, max_attempts + 1):
    print(f"\n--- Attempt {attempt}: Calling A2A Planner ---")
    print(f"    Objective: {current_objective}")

    # Build the A2A JSON-RPC request
    payload = {
        "jsonrpc": "2.0",
        "id": f"req-{attempt}",
        "method": "message/send",
        "params": {
            "message": {
                "message_id": str(uuid.uuid4()),
                "parts": [{"text": current_objective}],
                "role": "user"
            }
        }
    }

    try:
        async with httpx.AsyncClient(timeout=300.0) as client:
            resp = await client.post(
                f"{A2A_SERVER_URL}/", json=payload
            )
            data = resp.json()
            if "error" in data:
                final_report = data["error"].get("message", "Unknown A2A error")
            elif "result" in data:
                artifacts = data["result"].get("artifacts", [])
                if artifacts and "parts" in artifacts[-1]:
                    parts = artifacts[-1]["parts"]
                    if parts and "text" in parts[0]:
                        final_report = parts[0]["text"]
    except Exception as e:
        final_report = f"Connection error: {e}"

    print(f"\n--- Report ---\n{final_report}\n")
    is_success, should_retry = _classify_report(final_report)

    if is_success:
        yield Event(
            author=self.name,
            invocation_id=ctx.invocation_id,
            content=types.Content(
                role="model",
                parts=[types.Part.from_text(text=final_report)]
            ),
        )

```

```

        return

# --- Re-planning (CUJ 2) ---
if should_retry and attempt < max_attempts:
    print("--- Re-planning with LLM ---")
    replanner = LlmAgent(
        name=f"replanner_{attempt}",
        model="gemini-2.5-flash",
        instruction=(
            "You are a strategic re-planner. The previous request "
            "failed. Rewrite the objective to be broader or more "
            "likely to succeed. Output ONLY the new objective text."
        ),
    )

# Run the re-planner as a sub-agent
child_ctx = InvocationContext(
    invocation_id=f"{ctx.invocation_id}_replan_{attempt}",
    agent=replanner,
    session=ctx.session,
    session_service=ctx.session_service,
    run_config=ctx.run_config or RunConfig(),
)

child_ctx.user_content = types.Content(
    role="user",
    parts=[types.Part.from_text(text=(
        f"Original Objective: {current_objective}\n"
        f"Failure Reason: {final_report}\n"
        "Please broaden the search."
    ))]
)

new_objective = ""
async for event in replanner.run_async(child_ctx):
    if event.content and event.content.parts:
        for part in event.content.parts:
            if hasattr(part, "text") and part.text:
                new_objective += part.text

current_objective = new_objective.strip()
print(f"New objective: {current_objective}")
continue

# Terminal failure (not retryable)
if not should_retry:
    yield Event(
        author=self.name,
        invocation_id=ctx.invocation_id,
        content=types.Content(
            role="model",
            parts=[types.Part.from_text(text=f"FAILED: {final_report}")]
        ),
    )

```

```

        )
        return

    # Max attempts exhausted
    yield Event(
        author=self.name,
        invocation_id=ctx.invocation_id,
        content=types.Content(
            role="model",
            parts=[types.Part.from_text(
                text=f"FAILED after {max_attempts} attempts: {final_report}"
            )]
        ),
    )
)

async def main():
    prompt = "Restock 1 Pixel 7 phones for the Tokyo office"
    print(f"Starting Control Room with: {prompt}\n")

    agent = ControlRoomAgent(name="control_room")
    runner = InMemoryRunner(app_name="control_room", agent=agent)
    session = await runner.session_service.create_session(
        app_name="control_room", user_id="admin"
    )
    content = types.Content(
        role="user", parts=[types.Part.from_text(text=prompt)]
    )

    async for event in runner.run_async(
        user_id="admin", session_id=session.id, new_message=content
    ):
        if event.content and event.content.parts:
            for part in event.content.parts:
                if hasattr(part, "text") and part.text:
                    print(f"Result: {part.text}")

    print("\n=== CONTROL ROOM COMPLETE ===")

if __name__ == "__main__":
    asyncio.run(main())

```

核心概念

- **BaseAgent**：自訂代理的 ADK 基本元素。您可以將其子類別化並覆寫 `_run_async_impl`，以編寫任意非同步自動調度管理邏輯，例如 A2A 呼叫 + 分類 + 重新規劃迴圈。
- **A2A JSON-RPC 呼叫**：控制室會使用 `httpx` 將標準 `message/send` 要求傳送至規劃工具的 A2A 伺服器，並剖析 JSON-RPC 回應，以擷取最終報表。
- **_classify_report()**：簡單的關鍵字分類，可根據報表文字判斷成功、可重試的失敗或終端失敗。這會啟動重新規劃迴圈。

- **子代理程式叫用**：如要重新規劃，控制室會建立 `LlmAgent`，並建構子 `InvocationContext` 和呼叫 `replanner.run_async(child_ctx)` 來執行。您可以在自訂協調邏輯中動態啟動 LLM 代理程式。
 - **InMemoryRunner**：在本機執行代理程式，並使用記憶體內的工作階段儲存區。在正式環境中，您會使用 `adk deploy` 部署至 Vertex AI Agent Engine。
-

9. 執行完整堆疊

現在，讓我們測試完整的三層系統：ADK 控制室 → A2A → LangGraph Planner → CrewAI Crew。

使用先前開啟的第二個 Cloud Shell 分頁 (或點選「+」開啟新分頁)，然後執行 Control Room。**重要事項：**每個 Cloud Shell 分頁都有自己的 Shell 工作階段。您必須重新設定專案和環境變數：

```
cd ~/scale-agents
export GOOGLE_CLOUD_PROJECT=$(gcloud config get-value project)
export GOOGLE_CLOUD_LOCATION=us-central1
export GOOGLE_GENAI_USE_VERTEXAI=TRUE
uv run python control_room.py
```

您應該會看到完整的自動化流程：

- 1. 控制室會收到要求，並將 `message/send` JSON-RPC 呼叫傳送至 A2A 伺服器
- 2. A2A 伺服器會接收要求並叫用 LangGraph 規劃工具
- 3. LangGraph 規劃工具會分析要求，並委派給 CrewAI 團隊
- 4. CrewAI 團隊會執行採購代理程式
- 5. 結果會一路回傳至控制室

關鍵使用者歷程 (CUJ)

請試著修改 `control_room.py` 中的 `prompt` 字串，以實驗這些情境：

CUJ	提示詞	會發生什麼情況
1. 滿意路徑	<code>Restock 1 Pixel 7 phones for the Tokyo office</code>	搜尋 -> 預算檢查 -> 訂購單 (成功)。端對端運作。
2. 重新規劃	<code>Order 1 unit of the discontinued XR-7000 Quantum Holographic Display</code>	規劃師會傳回「失敗：不明項目」。控制室會偵測到這點，並叫用 <code>LlmAgent</code> 重新規劃工具來擴大搜尋範圍。兩次嘗試都會失敗 (硬式編碼的規劃工具只會辨識「Pixel 7」)，但您會看到完整的重新規劃機制運作。

如要測試 **CUJ 2 (重新規劃)**，請將 `control_room.py` 中的 `prompt` 變更為：

```
prompt = "Order 1 unit of the discontinued XR-7000 Quantum Holographic Display"
```

硬式編碼的規劃工具無法辨識這個項目，並會傳回「失敗：不明項目」。控制室會偵測到失敗，動態建立 `LlmAgent` 重新規劃工具，並以更廣泛的目標重試。由於規劃工具只會辨識「Pixel 7」，因此重試也會失敗，但您會看到完整的重新規劃迴圈運作。最終輸出內容會是 `FAILED after 2 attempts: ...`。

步驟 10 · 約 1 分鐘

10. 清除所用資源

如要避免系統持續向您的 Google Cloud 帳戶收取費用，請刪除本程式碼研究室建立的資源。您可以直接移除建立的目錄：

```
cd ..  
rm -rf scale-agents
```

11. 恭喜

恭喜！您已使用 CrewAI、LangGraph、A2A 通訊協定和 ADK，成功建構多代理自動化調度管理系統。

目前所學內容

- 如何使用 CrewAI 的 `@tool` 修飾符定義代理程式的工具。
- 如何建立具有不同角色、工具和目標的專業代理程式。
- 如何將代理程式串連到具有工作依附元件的序列團隊。
- 如何使用 LangGraph 建構狀態機規劃工具，將工作委派給工作人員。
- 如何使用 `AgentCard` 和 `AgentExecutor` 將規劃工具公開為 A2A 服務。
- 如何建構自訂 ADK `BaseAgent`，透過 A2A 委派工作，並在失敗時叫用 `LlmAgent` 子代理重新規劃。
- 瞭解為何將規劃、執行和自動化調度管理作業分散到各個架構，可讓您享有模組化和彈性。

進一步瞭解

完整研討會將透過下列方式擴充這個系統：

- **即時資訊主頁**：SSE 串流，可將多個代理程式的進度視覺化
- **身分防護**：以 IAM 為基礎的安全性，可在基礎架構層級而非提示層級阻擋破壞性動作
- **Vertex AI Agent Engine**：將 ADK 代理部署至代管雲端基礎架構，並使用 `adk deploy`

參考文件

- [CrewAI 說明文件](#)
 - [LangGraph 說明文件](#)
 - [Vertex AI 說明文件](#)
 - [A2A 通訊協定](#)
 - [ADK 說明文件](#)
-